DSA03 – NATURAL LANGUAGE PROCESSING

ASSIGNMENT – 1 (CO1)

Assignment Title: Build a Morphology-Aware Text Normalisation and Search Layer for a Hospital Record System Using Regular Expressions, Finite-State Automata and Morphological Parsing

Student Name: TELAPROLU ROHITA SAI
Register Number: 192425041
Department: Artificial Intelligence and Machine Learning
Course Code: DSA03
Course Name: Natural Language Processing
Total Marks: 50

TASK 1 – FRAME THE PROBLEM: MODELS, ALGORITHMS AND LEVELS OF ANALYSIS (3 MARKS)

1.1 Introduction

Natural Language Processing (NLP) helps computers understand, process and work with human language. In a hospital, thousands of medical records are stored as text, including prescriptions, discharge summaries, operation notes and nursing observations.

Searching these records using only exact keywords can be difficult. For example, when a doctor searches for the word "fracture", the system may fail to retrieve records containing "fractures", "fractured" or "fracturing".

To solve this problem, I propose a text normalisation and morphological analysis system that converts different forms of clinical words into standard representations. It uses regular expressions, finite-state automata, morphological parsing and stemming to improve the accuracy of hospital record searches.

1.2 Levels of Linguistic Analysis

NLP involves different levels of language analysis. Each level handles a particular aspect of human language.

Level	Description	Hospital Example	Requirement
Phonology	Studies speech sounds and pronunciation.	Understanding spoken medical terms.	Not required for typed records.
Morphology	Studies how words are formed using roots and affixes.	Fractures → fracture + plural.	Required.
Syntax	Studies sentence structure and grammatical relationships.	"Patient was given paracetamol."	Partially required.
Semantics	Studies the meaning of words and sentences.	Understanding whether a patient has or denies fever.	Required for advanced clinical interpretation.
Pragmatics	Studies meaning based on context and intention.	Understanding a doctor's instruction in a particular situation.	Not essential for basic normalisation.

Discourse	Studies relationships between sentences across a document.	Connecting a diagnosis mentioned in a discharge summary with its treatment.	Useful for advanced searching.

Interpretation:

For this project, morphology is one of the most important levels because the same medical term can appear in different grammatical forms. Regular expressions and tokenisation handle the initial text processing, while morphology connects related word forms.

Basic syntax, semantics and discourse analysis can improve the system further, but they are beyond the main objective of this finite-state normalisation layer.

1.3 Three Model Families

The three major model families used in NLP are state machines, formal rule systems and probabilistic models.

Model Family	Description	Suitable Hospital Task	Why Other Models Are Less Suitable
State machines	Process input through a sequence of states and transitions.	Validating patient IDs and dates.	Probabilistic models may produce uncertain validation results.
Formal rule systems	Apply predefined linguistic and spelling rules.	Expanding abbreviations and handling word suffixes.	State machines alone do not conveniently express every linguistic transformation.
Probabilistic models	Use probabilities learned from data to make predictions.	Correcting misspelt medical terms.	Fixed rules cannot easily handle every possible spelling variation.

For example, a state machine can verify whether a patient ID follows a fixed format. A rule-based system can convert "fractures" into "fracture". A probabilistic model can suggest the intended word when OCR produces an incorrect spelling.

1.4 Ambiguity in Natural Language Processing

Ambiguity occurs when a word or sentence has more than one possible interpretation. It is one of the major challenges in NLP.

The hospital dataset may contain the following types of ambiguity:

1. **Lexical ambiguity:** A word can have different meanings depending on context. For example, "discharge" may refer to a hospital discharge or bodily discharge.
2. **Morphological ambiguity:** A word may have more than one possible grammatical analysis. For example, "treated" may be interpreted as a past-tense verb or a participial adjective.
3. **Segmentation ambiguity:** The same characters may be divided into tokens differently. For example, "500mg" may be treated as one token or as "500" and "mg".
4. **Abbreviation ambiguity:** A short form may represent different medical terms. For example, "OD" can mean once daily in a prescription or right eye in an ophthalmic context.

These examples show why the system must consider both word structure and the context in which words appear.

1.5 Search in NLP and Its Resources

Search is used in finite-state automata to identify valid paths through states. Backtracking is used in pattern matching when the system tries different possible matching paths.

Resource	Usage
Time	Required to scan records, match patterns and process words.
Memory	Required to store automata, lexicons and intermediate results.
Annotated data	Required to evaluate the accuracy of the system.
Expert effort	Required to prepare medical vocabulary, patterns and annotations.

Regular expressions and finite-state automata are efficient for structured text, but building and maintaining medical rules requires manual effort.

1.6 Proposed System Pipeline

The proposed system follows a sequence of processing stages.

Pipeline Diagram:

Raw Hospital Records
↓
Text Cleaning and Preprocessing
↓ *(Regular Expressions)*
Tokenisation
↓ *(Regex + Maximal-Munch)*

Entity Recognition
↓ *(Regular Expressions + Finite-State Automata)*
Text Normalisation
↓ *(Substitution Rules)*
Morphological Analysis
↓ *(Finite-State Transducer)*
Stemming
↓ *(Porter Stemmer)*
Search Index Creation
↓
Normalised Search Results

Input: Raw clinical records containing medical terms, patient details, dates and prescriptions.

Output: Normalised and searchable hospital records with extracted entities and morphological information.

Conclusion:

The proposed pipeline combines different NLP techniques instead of depending on a single method. This makes it easier to recognise structured information, process different word forms and improve clinical information retrieval.

TASK 2 – BASIC REGULAR EXPRESSION PATTERNS (4 MARKS)

2.1 Introduction to Regular Expressions

Regular Expressions (Regex) are patterns used to identify and manipulate specific sequences of characters in text.

In the hospital system, regular expressions help recognise patient identifiers, dates, drug dosages, laboratory results and other structured information.

2.2 Basic Regular Expression Operators

Operator	Meaning	Clinical Example
Concatenation	Joins characters or patterns in sequence.	AB123
Disjunction |	Matches either of two alternatives.	male|female
Kleene Star *	Matches zero or more repetitions.	mg*
Plus +	Matches one or more repetitions.	A+
Optional ?	Matches zero or one occurrence.	kg?
Character Class []	Matches one character from a set.	[0-9]
Range -	Matches a range of characters.	[A-Z]
Negated Class [^]	Matches characters not included in the class.	[^0-9]
Anchors ^ and $	Match the beginning and end of a string.	^PAT
Counters {}	Specify the number of repetitions.	[0-9]{4}

Examples

- [0-9]+ matches one or more digits.
- [A-Z]{2}[0-9]{4} matches two uppercase letters followed by four digits.
- mg|ml matches either mg or ml.
- colou?r matches both colour and color.

2.3 Regular Expressions for 12 Clinical Entity Types

The following patterns are designed for the proposed hospital record system using Python regular expression syntax.

Entity	Regular Expression	Example
Patient ID	\bPAT[0-9]{4,8}\b	PAT12345
Admission Date	\b(?:0[1-9]|[12][0-9]|3[01])[-/](?:0[1-9]|1[0-2])[-/](?:20)[0-9]{2}\b	12/04/2026
Discharge Date	Same date pattern	15-04-2026
Drug with Strength	\b[A-Za-z][A-Za-z-]*\s+[0-9]+(?:\.[0-9]+)?\s*(?:mg|g|mcg|ml)\b	Paracetamol 500mg
Dosage Frequency	\b(?:OD|BD|TDS|QID|BID)\b	TDS
Laboratory Value	\b[A-Za-z][A-Za-z-]*\s*[:=]?\s*[0-9]+(?:\.[0-9]+)?\s*(?:mg/dL|g/dL|mmol/L|U/L)\b	Glucose 120 mg/dL
Blood Pressure	\b[0-9]{2,3}/[0-9]{2,3}\s*mmHg\b	120/80 mmHg
Temperature	\b(?:3[0-9](?:\.[0-9]+)?|4[0-2](?:\.[0-9]+)?)\s*°?C\b	37.5 C
ICD Code	\b[A-TV-Z][0-9][A-Z0-9](?:\.[A-Z0-9]{1,4})?\b	J18.9
Ward and Bed	\bWard\s*[A-Z0-9-]+\s*,?\s*Bed\s*[A-Z0-9-]+\b	Ward A, Bed 12
Contact Number	\b(?:\+91[-]?)?[6-9][0-9]{9}\b	9876543210
Age	`\b(?:[1-9][0-9]? | 1[01][0-9] | 120)\s*(??|yrs?)\b`	45 years
Symptom Duration	\b[0-9]+\s*(?:hours?|days?|weeks?|months?)\b	3 days		

What the Patterns Must Match and Must Not Match

1. Patient ID must match PAT followed by 4–8 digits. It must reject incomplete IDs such as PAT12.
2. Dates must match valid-looking day, month and year formats. They must reject invalid months such as 19.
3. Drug strength must contain a drug name, numerical strength and recognised unit. It must reject unrelated numbers.
4. Dosage frequency must match recognised frequency abbreviations. It must reject random character combinations.
5. Laboratory values must contain a test name, numerical value and accepted unit. They must reject values without units.
6. Blood pressure must contain two numerical readings separated by a slash and followed by mmHg. It must reject 120-80.
7. Temperature must contain a valid-looking numerical value and Celsius unit. It must reject unrelated numbers.
8. ICD codes must follow the expected alphanumeric structure. They must reject incomplete codes.
9. Ward and bed must contain both a ward and a bed identifier. It must reject a ward name without a bed.
10. Contact numbers must contain a valid-looking mobile number. They must reject numbers with fewer than ten digits.
11. Age must contain a numerical age and a recognised age unit. It must reject negative ages.
12. Symptom duration must contain a number and a recognised time unit. It must reject text without a duration.

2.4 Greedy and Non-Greedy Matching

Greedy matching attempts to capture the longest possible match, while non-greedy matching captures the shortest possible match that satisfies the pattern.

Clinical Sentence:

"Patient was prescribed Paracetamol 500mg and Ibuprofen 200mg."

Greedy pattern:

<escape>.*mg</escape>

It may capture:

"Patient was prescribed Paracetamol 500mg and Ibuprofen 200mg"

Non-greedy pattern:

<escape>.*?mg</escape>

It captures the shortest matching span ending at the first occurrence of mg.

For identifying individual dosage expressions, a better pattern is:

[A-Za-z][A-Za-z-]*\s+[0-9]+mg

This pattern identifies each dosage separately.

Interpretation: Greedy matching is useful when the complete longest span is required. Non-greedy matching is useful when the system needs to stop at the earliest possible match.

2.5 Operator Precedence and Grouping

Regular expression operators have different precedence levels. Repetition operators are generally applied before concatenation, and concatenation before alternation.

Consider:

PAT|DOC[0-9]+

This expression means either PAT or DOC followed by one or more digits.

Therefore, it may incorrectly accept PAT without any digits.

The corrected expression is:

(?:PAT|DOC)[0-9]+

Here, grouping ensures that both PAT and DOC must be followed by digits.

2.6 Evaluation of Patterns

The patterns should be evaluated using manually labelled clinical records.

Metric	**Meaning**
True Positive (TP)	Correctly identified clinical entities.
False Positive (FP)	Incorrectly identified text.

False Negative (FN)	Valid entities missed by the pattern.

Precision:

Precision = TP / (TP + FP)

Recall:

Recall = TP / (TP + FN)

F1 Score:

F1 = 2 × Precision × Recall / (Precision + Recall)

Possible error causes include OCR mistakes, unusual date formats, misspelt drug names, missing spaces and unrecognised abbreviations.

Conclusion:

Regular expressions provide an efficient way to recognise structured clinical information. However, they must be carefully designed and tested because real medical records may contain inconsistent formats and spelling variations.

TASK 3 – REGULAR EXPRESSIONS FOR TOKENISATION AND NORMALISATION (4 MARKS)

3.1 Introduction

Tokenisation is the process of dividing a sentence into smaller units called tokens. These tokens may be words, numbers, abbreviations or medical expressions.

Normalisation converts different forms of the same information into a consistent format.

For example:

"PARACETAMOL 500 MG" and "Paracetamol 500mg" can be converted into a standard representation.

3.2 Tokenisation Regular Expression

The following Python regular expression is used to recognise common clinical tokens.

```
(?:[A-Za-z]+(?:-[A-Za-z]+)*(?:\.[A-Za-z]+)*|[0-9]+(?:\.[0-
9]+)?(?:mg/dL|g/dL|mmHg|mmol/L|mg|mcg|ml|kg|°C)?|[^\s])
```

It recognises:

- Ordinary words.
- Hyphenated medical terms.
- Abbreviations containing full stops.
- Decimal numbers.
- Numbers attached to medical units.
- Individual punctuation marks.

The tokenisation decision is based on preserving important clinical information instead of splitting every expression at whitespace.

3.3 Maximal-Munch Tokenisation

Maximal-munch means selecting the longest valid token at each position.

Clinical Text	Naive Whitespace Split	Maximal-Munch Result
Paracetamol 500mg daily	Paracetamol, 500mg, daily	Paracetamol, 500mg, daily
BP 120/80 mmHg	BP, 120/80, mmHg	BP, 120/80, mmHg
Post-operative care	Post-operative, care	Post-operative, care

37.5°C recorded	37.5°C, recorded	37.5°C, recorded
Dr. Kumar examined	Dr., Kumar, examined	Dr., Kumar, examined

Whitespace splitting sometimes produces the same result, but it does not understand token boundaries. A dedicated tokenizer can also preserve punctuation, abbreviations and numerical units consistently.

3.4 Text Normalisation Rules

The following rules are used to standardise hospital records.

Normalisation	Before	After
Case folding	FEVER	fever
Abbreviation expansion	BP	blood pressure
Abbreviation expansion	HR	heart rate
Abbreviation expansion	TDS	three times daily
Unit standardisation	500 MG	500 mg
Date standardisation	12/04/2026	2026-04-12
Whitespace removal	Multiple spaces	Single space
Word normalisation	Fractures	fracture

Clinical Abbreviations

The system can maintain a dictionary containing at least 30 common abbreviations.

Abbreviation	Expanded Form
BP	Blood Pressure
HR	Heart Rate
RR	Respiratory Rate
Temp	Temperature
SpO2	Oxygen Saturation

WBC	White Blood Cell
RBC	Red Blood Cell
Hb	Haemoglobin
CBC	Complete Blood Count
ECG	Electrocardiogram
CT	Computed Tomography
MRI	Magnetic Resonance Imaging
USG	Ultrasonography
IV	Intravenous
IM	Intramuscular
PO	By Mouth
OD	Once Daily
BD	Twice Daily
TDS	Three Times Daily
QID	Four Times Daily
PRN	As Needed
SOS	If Necessary
ICU	Intensive Care Unit
OPD	Outpatient Department
IPD	Inpatient Department
ENT	Ear, Nose and Throat
UTI	Urinary Tract Infection
DM	Diabetes Mellitus

HTN	Hypertension
SOB	Shortness of Breath

Some abbreviations have multiple meanings. Therefore, the system should use surrounding words and record type before expanding ambiguous abbreviations.

3.5 Before-and-After Clinical Record

Original Record:

"PAT12345 admitted on 12/04/2026 with FEVER for 3 days. BP 120/80 mmHg. Hb 12.5 g/dL. Paracetamol 500 MG given TDS. Patient discharged on 15/04/2026."

Normalised Record:

"pat12345 admitted on 2026-04-12 with fever for 3 days. blood pressure 120/80 mmHg. haemoglobin 12.5 g/dL. paracetamol 500 mg given three times daily. patient discharged on 2026-04-15."

This standardised format makes the record easier to search and compare.

3.6 Irreversible Normalisation Decisions

Some normalisation operations permanently remove information.

1. **Case folding:** Converting all text to lowercase removes the original capitalisation.
2. **Abbreviation expansion:** Replacing an abbreviation with its expanded form may remove the exact original wording.
3. **Punctuation removal:** Removing punctuation can change the meaning of certain medical expressions.
4. **Whitespace removal:** Joining tokens incorrectly may destroy word boundaries.
5. **Unit conversion:** Converting measurements may introduce rounding or precision loss.

These changes are acceptable when they improve search consistency. However, original clinical records must always be preserved because even small changes may affect medical interpretation.

3.7 Limitations of Regular Expressions

Regular expressions are suitable for recognizing patterns with predictable structures. However, they cannot conveniently handle every language requirement.

One example is matching balanced nested parentheses.

Consider:

Patient note (history (previous surgery))

A regular expression based on ordinary finite-state matching cannot recognise arbitrarily nested balanced parentheses because this requires remembering an unlimited nesting depth.

A context-free grammar or stack-based parser is more suitable for this task.

Conclusion:

Regular expressions are useful for tokenization, structured entity extraction and basic text normalization. However, they cannot fully understand language meaning or handle unlimited nested structures. Therefore, they should be combined with morphological and contextual analysis.

TASK 4 – FINITE STATE AUTOMATA: DESIGN, NON-DETERMINISM AND EQUIVALENCE (5 MARKS)

4.1 Introduction

A Finite State Automaton (FSA) is a mathematical model used to recognise whether an input string belongs to a particular language.

It consists of a finite number of states and transitions. When an input character is received, the automaton moves from one state to another according to its transition function.

In the hospital system, automata can validate patient IDs, dates, blood-pressure readings and contact numbers.

4.2 Formal Definition of Finite Automaton

A finite automaton is represented by the five-tuple:

$M = (Q, \Sigma, \delta, q_0, F)$

Where:

- Q = Finite set of states.
- Σ = Input alphabet.
- δ = Transition function.
- q_0 = Initial state.
- F = Set of final or accepting states.

A string is accepted when the automaton finishes in an accepting state after reading the complete input.

4.3 DFA 1 – Patient ID Validation

Pattern: PAT[0-9]{4}

Example: PAT1234

States:

- q0 – Initial state.
- q1 – P received.
- q2 – A received.
- q3 – T received.
- q4 – First digit received.

- q5 – Second digit received.
- q6 – Third digit received.
- q7 – Fourth digit received; accepting state.

Transition Diagram:

q0 --P--> q1

q1 --A--> q2

q2 --T--> q3

q3 --Digit--> q4

q4 --Digit--> q5

q5 --Digit--> q6

q6 --Digit--> q7*

The automaton accepts PAT followed by exactly four digits.

Transition Table:

State	Input	Next State
q0	P	q1
q1	A	q2
q2	T	q3
q3	Digit	q4
q4	Digit	q5
q5	Digit	q6
q6	Digit	q7
q7	Any additional character	Reject

Accepting state: q7.

4.4 DFA 2 – Contact Number Validation

Pattern: [6-9][0-9]{9}

Example: 9876543210

The automaton first checks whether the number begins with a digit from 6 to 9. It then accepts exactly nine additional digits.

States:

q0 – Initial state.

q1 – Valid first digit received.

q2 – Second digit received.

q3 – Third digit received.

q4 – Fourth digit received.

q5 – Fifth digit received.

q6 – Sixth digit received.

q7 – Seventh digit received.

q8 – Eighth digit received.

q9 – Ninth digit received.

q10 – Tenth digit received; accepting state.

Transition Diagram:

q0 --[6-9]--> q1

q1 --[0-9]--> q2

q2 --[0-9]--> q3

q3 --[0-9]--> q4

q4 --[0-9]--> q5

q5 --[0-9]--> q6

q6 --[0-9]--> q7

q7 --[0-9]--> q8

q8 --[0-9]--> q9

q9 --[0-9]--> q10*

Accepting state: q10.

4.5 DFA 3 – Blood Pressure Validation

Pattern: [0-9]{2,3}/[0-9]{2,3} mmHg

Example: 120/80 mmHg

The automaton recognises two or three digits, a slash, two or three digits, a space and the unit mmHg.

States:

- q0 – Initial state.
- q1 – First reading begins.
- q2 – First reading complete.
- q3 – Slash received.
- q4 – Second reading begins.
- q5 – Second reading complete.
- q6 – Space received.
- q7 – m received.
- q8 – m received.
- q9 – H received.
- q10 – g received; accepting state.

The numeric portions require additional counting states to enforce the minimum and maximum digit lengths.

4.6 DFA 4 – Date Validation

Pattern: DD/MM/YYYY

Example: 12/04/2026

The automaton reads two digits for the day, a slash, two digits for the month, another slash and four digits for the year.

Transition Diagram:

q0 --Digit--> q1

q1 --Digit--> q2

q2 --/--> q3

q3 --Digit--> q4

q4 --Digit--> q5

q5 --/--> q6

q6 --Digit--> q7

q7 --Digit--> q8

q8 --Digit--> q9

q9 --Digit--> q10*

Accepting state: q10.

The DFA checks the structure of the date. A separate calendar-validation function is required to check the actual number of days in each month and leap years.

4.7 Non-Deterministic Finite Automaton (NFA)

An NFA allows more than one possible transition from a state for the same input. It can also contain epsilon transitions, which allow movement without consuming an input character.

Consider the patient ID format:

PAT followed by an optional hyphen and four digits.

Regular Expression:

PAT-?[0-9]{4}

The NFA can have two possible paths after reading PAT:

1. Continue directly to the digits.
2. Consume a hyphen before reading the digits.

Transition Description:

q0 --P--> q1

q1 --A--> q2

q2 --T--> q3

q3 --ε--> q4

q3 --- --> q5

q4 --Digit--> q6

q5 --Digit--> q6

q6 --Digit--> q7

q7 --Digit--> q8*

The accepting state is q8.

Accepted String: PAT-1234

Rejected String: PAT-12A4

The rejected string fails because the third expected digit is replaced by a letter.

4.8 NFA to DFA Conversion Using Subset Construction

Subset construction converts an NFA into an equivalent DFA by treating sets of NFA states as individual DFA states.

The process involves:

1. Finding the epsilon closure of the initial state.
2. Calculating transitions for each input symbol.
3. Finding the epsilon closure of every resulting state set.
4. Repeating the process until no new state sets are generated.
5. Identifying accepting sets containing an NFA accepting state.

For the above NFA, the epsilon closure of q3 contains q3 and q4.

The resulting DFA has separate states for the path with a hyphen and the path without a hyphen. Both paths eventually reach an accepting state after four digits.

The exact number of DFA states depends on the state representation and treatment of the hyphen transition.

4.9 DFA Minimisation

DFA minimisation reduces the number of states without changing the language accepted by the automaton.

The process includes:

1. Separating accepting and non-accepting states.
2. Comparing states based on their transitions.

3. Dividing states into equivalent groups.
4. Merging equivalent states.
5. Creating the final minimum DFA.

Minimisation reduces memory requirements and simplifies the automaton.

4.10 Thompson's Construction

Thompson's construction converts a regular expression into an equivalent NFA using basic constructions for concatenation, union and repetition.

For example:

PAT[0-9]{4}

It is constructed by joining the individual character automata in sequence.

Each literal character has a transition from an initial state to a final state. Concatenation connects the final state of one component to the initial state of the next component.

4.11 Regular Expression from an Automaton

A regular expression can also be obtained from an automaton using state elimination.

During state elimination, intermediate states are removed while the regular expression on the remaining transitions is updated to preserve all possible paths.

For the patient ID automaton:

PAT[0-9][0-9][0-9][0-9]

This expression represents the same language as the patient ID DFA.

Thus, regular expressions and finite-state automata can describe the same regular languages.

4.12 Pumping Lemma

The pumping lemma states that if a language is regular, there exists a pumping length p such that every sufficiently long string in the language can be divided into three parts:

w = xyz

Subject to:

1. $|xy| \leq p$
2. $|y| > 0$
3. xy^iz belongs to the language for every $i \geq 0$.

Consider the language of balanced parentheses:

$L = \{ (^n)^n \mid n \geq 0 \}$

Assume that this language is regular and has pumping length p.

Choose:

$w = (^p)^p$

Because $|xy| \leq p$, the substring y contains only opening parentheses.

Since $|y| > 0$, pumping y with $i = 0$ removes some opening parentheses while leaving the closing parentheses unchanged.

The resulting string has unequal numbers of opening and closing parentheses and is not balanced.

This contradicts the pumping lemma.

Therefore, the language of arbitrarily balanced parentheses is not regular.

Conclusion:

Finite-state automata are useful for validating structured clinical information. Regular expressions and automata are equivalent in their ability to describe regular languages. However, nested structures require more powerful computational models.

TASK 5 – INFLECTIONAL MORPHOLOGY (3 MARKS)

5.1 Introduction

Inflectional morphology studies how words change their forms to express grammatical information such as tense, number, person and comparison without changing their basic meaning or grammatical category.

In hospital records, the same medical word may appear in different forms. For example, "fracture", "fractures" and "fractured" are related forms that should be connected during searching.

5.2 Basic Morphological Terms

Term	Definition	Example
Morpheme	Smallest meaningful unit of a word.	Un + kind
Stem	Main part of a word to which an affix can be attached.	Treat
Affix	A group of letters added to a stem.	-ed
Inflection	Changes the grammatical form of a word.	Treat → treated
Paradigm	Set of grammatical forms of a word.	Treat, treats, treated, treating

5.3 Difference Between Inflection and Derivation

Inflection	Derivation
Changes grammatical information.	Creates a new word or meaning.
Usually preserves the grammatical category.	May change the grammatical category.
Generally more productive.	Productivity varies by affix.
Usually appears after derivational suffixes.	Can appear before inflectional suffixes.
Example: fracture → fractures.	Example: operate → operation.

5.4 Inflectional Paradigms in Clinical English

The following paradigms are commonly found in clinical records.

1. Noun Number

- Fracture → Fractures

- Infection → Infections
- Patient → Patients

2. Verb Tense

- Treat → Treated
- Admit → Admitted
- Examine → Examined

3. Verb Aspect

- Treat → Treating
- Monitor → Monitoring
- Operate → Operating

4. Subject Agreement

- Patient has a fever.
- Patients have a fever.

5. Adjective Comparison

- High → Higher → Highest
- Low → Lower → Lowest

5.5 Full Paradigms of Five Clinical Lexemes

Base Word	Singular / Base	Plural / Third Person	Past Form	Continuous Form
Fracture	Fracture	Fractures	Fractured	Fracturing
Treat	Treat	Treats	Treated	Treating
Admit	Admit	Admits	Admitted	Admitting
Monitor	Monitor	Monitors	Monitored	Monitoring
Examine	Examine	Examines	Examined	Examining

These examples demonstrate how inflection creates different grammatical forms while retaining the basic lexical identity.

5.6 English Inflectional Spelling Rules

English inflection involves spelling changes when suffixes are attached to words.

1. E-deletion Rule

When a word ends in silent e, the e is generally removed before adding -ing.

Example:

- Examine → Examining
- Operate → Operating

Counter-example:

- Be → Being

The e is retained in this irregular spelling.

2. Consonant Doubling Rule

For certain short words ending in consonant-vowel-consonant patterns, the final consonant is doubled before adding -ed or -ing.

Example:

- Admit → Admitted
- Admit → Admitting

Counter-example:

- Open → Opened

The final consonant is not doubled because the word does not follow the required stress pattern.

3. Y-to-I Rule

When a consonant comes before final y, the y generally changes to i before adding certain suffixes.

Example:

- Carry → Carried
- Study → Studies

Counter-example:

- Play → Played

The y is retained when a vowel occurs before it.

4. Sibilant -es Rule

Words ending in s, x, z, ch or sh generally take -es in the third-person singular or plural forms.

Example:

- Watch → Watches
- Dress → Dresses
- Box → Boxes

Counter-example:

- Treat → Treats

The word does not require -es.

5.7 Irregular Forms

Some English words do not follow regular spelling rules. These forms must be stored separately in the lexicon.

Base Form	Irregular Form
Be	Was, were, been
Have	Has, had
Go	Went, gone
Do	Did, done
Take	Took, taken
Give	Gave, given
Write	Wrote, written
See	Saw, seen
Run	Ran, run
Make	Made, made

These forms cannot always be generated using simple suffix rules. Therefore, an irregular-form dictionary is required.

5.8 Type-to-Lemma Ratio

The type-to-lemma ratio measures the number of different word forms compared with the number of unique lemmas.

Formula:

Type-to-Lemma Ratio = Number of Unique Word Forms / Number of Unique Lemmas

For example, consider the following word forms:

- Treat
- Treats
- Treated
- Treating
- Monitor
- Monitors

There are six unique word forms and two lemmas.

Type-to-Lemma Ratio = 6 / 2 = 3.

A higher ratio indicates that the same basic words appear in multiple forms.

The actual ratio for the hospital corpus must be calculated after processing the required clinical records.

5.9 Effect of Inflection on Retrieval

Inflection can affect search accuracy when the user searches using only one form of a word.

For example:

A search for "fracture" may fail to retrieve "fractures" or "fractured" if exact matching is used.

Morphological normalisation converts these related forms into a common representation and improves retrieval.

The proportion of queries affected by inflection can be calculated as:

Inflection Failure Rate = Queries Failed Due to Inflection / Total Queries × 100

Conclusion:

Inflectional morphology helps connect grammatical variations of clinical words. By applying spelling rules and storing irregular forms, the system can improve search coverage and reduce missed records.

TASK 6 – DERIVATIONAL MORPHOLOGY (4 MARKS)

6.1 Introduction

Derivational morphology studies how new words are formed by adding prefixes and suffixes to existing words.

Unlike inflection, derivation can change the meaning or grammatical category of a word.

For example:

- Operate → Operation
- Infect → Infection
- Normal → Abnormal

In medical vocabulary, derivational morphology is important because many specialised terms are formed using Greek and Latin roots and affixes.

6.2 Derivational Affixes in Clinical Vocabulary

Affix	Type	Meaning	Example
-itis	Suffix	Inflammation	Arthritis
-ectomy	Suffix	Surgical removal	Nephrectomy
-ology	Suffix	Study of	Cardiology
-osis	Suffix	Condition or disease	Fibrosis
-aemia	Suffix	Blood condition	Anaemia
-ation	Suffix	Process or action	Examination
-able	Suffix	Capable of	Treatable
-ive	Suffix	Having a quality	Preventive
-al	Suffix	Related to	Renal
-ic	Suffix	Related to	Cardiac
nephro-	Prefix	Kidney	Nephrology
cardio-	Prefix	Heart	Cardiology

hyper-	Prefix	Excessive or high	Hypertension
hypo-	Prefix	Low or below normal	Hypoglycaemia
re-	Prefix	Again	Reassessment
un-	Prefix	Not or opposite	Unstable

These affixes help create specialised clinical terminology.

6.3 Derivational Families of Six Clinical Roots

A derivational family contains words formed from the same root using different affixes.

1. Infect

- Infect
- Infection
- Infectious
- Infected
- Infective

2. Treat

- Treat
- Treatment
- Treatable
- Untreated
- Retreatment

3. Operate

- Operate
- Operation
- Operative
- Operated
- Inoperative

4. Diagnose

- Diagnose
- Diagnosis
- Diagnostic
- Diagnostically

5. Renal / Nephro

- Renal
- Nephrology
- Nephrologist
- Nephrectomy
- Nephritis

6. Cardiac / Cardio

- Cardiac
- Cardiology
- Cardiologist
- Cardiovascular
- Carditis

Some related terms are formed from shared roots rather than by directly attaching an affix to the complete modern English word. A medical lexicon must therefore store the appropriate root and word-family relationships.

6.4 Order of Affix Attachment

The order in which affixes are attached can change the grammatical structure and meaning of a word.

Examples:

Treat + able = Treatable

Un + treatable = Untreatable

Operate + ion = Operation

Prevent + ion = Prevention

Some combinations are grammatically incorrect.

Examples:

- -able + un- attached as an arbitrary reversed suffix sequence is not a valid construction for "untreatable".
- treat + -ing + -ed does not produce a normal English word.

The valid construction is determined by the rules of the language and the continuation classes of the lexicon.

6.5 Semantic Non-Compositionality

Semantic non-compositionality occurs when the meaning of a complete word cannot be predicted simply by combining the meanings of its individual parts.

Examples:

1. **Inflammation:** The term refers to a biological response and not simply the general meaning of its components.
2. **Nephrectomy:** It refers specifically to surgical removal of a kidney.
3. **Anaemia:** It refers to a medical condition involving reduced haemoglobin or red blood cells, depending on the clinical definition.

A search system that stems words too aggressively may connect terms that are linguistically related but clinically different.

Therefore, medical vocabulary should be preserved carefully instead of applying unrestricted stemming.

6.6 Productivity of Derivational Affixes

Productivity refers to how freely an affix can be used to create new words.

Affix	**Example**	**Productivity**
-able	Treatable	Productive
-ation	Examination	Productive
re-	Reassessment	Productive
un-	Unstable	Productive
-ive	Preventive	Productive
-itis	Arthritis	Relatively restricted

-ectomy	Nephrectomy	Restricted
-ology	Cardiology	Relatively productive
-osis	Fibrosis	Restricted
-aemia	Anaemia	Restricted

Actual productivity should be measured by counting unique word types and total occurrences of each affix in the clinical corpus.

A high number of different words formed with an affix suggests greater productivity.

6.7 Should Derivational Relatives Be Conflated for Search?

Conflation means grouping different word forms or related words under a common search representation.

Cases Where Conflation Helps:

1. Nephrectomy and nephrectomies can be connected to retrieve records involving the same procedure.
2. Infection and infections can be grouped to improve search coverage.

Cases Where Conflation Can Be Misleading:

1. Hypertension and hypotension are opposite medical conditions. Removing their prefixes could create an incorrect match.
2. Nephrectomy and nephritis are related to the kidney but refer to different medical conditions.

Therefore, derivational conflation should be selective. Words with closely related meanings can be grouped, but clinically distinct terms must remain separate.

Conclusion:

Derivational morphology helps the system understand specialised medical vocabulary. However, not all related words have the same meaning. A medical search system should combine morphological rules with a carefully maintained clinical lexicon.

TASK 7 – FINITE-STATE MORPHOLOGICAL PARSING (5 MARKS)

7.1 Introduction

Finite-State Morphological Parsing is a technique used to analyse the internal structure of words using a Finite-State Transducer (FST).

An FST identifies the relationship between the surface form of a word and its lexical representation.

For example:

Surface Form: fractures

Lexical Representation: fracture + N + PL

The system identifies the stem, grammatical category and morphological features.

7.2 Definition of Finite-State Transducer

A Finite-State Transducer is a two-tape automaton that reads symbols from one tape and produces corresponding symbols on another tape.

The two tapes are:

1. **Lexical Tape:** Contains the root word and grammatical features.
2. **Surface Tape:** Contains the actual written word.

For example:

Lexical Tape: fracture + N + PL

↓

FST

↓

Surface Tape: fractures

The same transducer can work in both directions:

- Analysis: Surface form → Lexical representation.
- Generation: Lexical representation → Surface form.

7.3 Building the Morphological Lexicon

The lexicon stores stems, their grammatical categories and possible affixes.

Example lexicon:

Stem	Category
Fracture	Noun
Infection	Noun
Patient	Noun
Treat	Verb
Admit	Verb
Monitor	Verb
Normal	Adjective
Severe	Adjective
Cardiac	Adjective
Renal	Adjective

The complete assignment requires at least 300 stems. These should be collected from the clinical vocabulary used in the corpus.

7.4 Continuation Classes

Continuation classes describe which suffixes or grammatical features can follow a particular lexical category.

Continuation Class	Possible Forms
Noun	+SG, +PL
Verb	+BASE, +3SG, +PAST, +ING
Adjective	+POS, +COMP, +SUPER
Irregular Verb	Stored irregular forms

For example:

fracture + N + PL → fractures

treat + V + PAST → treated

monitor + V + ING → monitoring

high + ADJ + COMP → higher

7.5 Hand Tracing of Five Surface Forms

1. Fractures

- Input: fractures
- Stem: fracture
- Category: Noun
- Feature: Plural
- Analysis: fracture + N + PL

2. Patients

- Input: patients
- Stem: patient
- Category: Noun
- Feature: Plural
- Analysis: patient + N + PL

3. Treated

- Input: treated
- Stem: treat
- Category: Verb
- Feature: Past tense
- Analysis: treat + V + PAST

4. Monitoring

- Input: monitoring
- Stem: monitor
- Category: Verb
- Feature: Present participle
- Analysis: monitor + V + ING

5. Infections

- Input: infections
- Stem: infection
- Category: Noun
- Feature: Plural
- Analysis: infection + N + PL

Transducer Processing

Input	Lexical Stem	Affix / Feature	Final Analysis
Fractures	Fracture	+N +PL	fracture +N +PL
Patients	Patient	+N +PL	patient +N +PL
Treated	Treat	+V +PAST	treat +V +PAST
Monitoring	Monitor	+V +ING	monitor +V +ING
Infections	Infection	+N +PL	infection +N +PL

7.6 Generation Using the Same Transducer

The same FST can generate surface forms from lexical representations.

Lexical Input	Generated Surface Form
fracture +N +PL	fractures
patient +N +PL	patients
treat +V +PAST	treated
monitor +V +ING	monitoring
infection +N +PL	infections

For example:

Input: fracture +N +PL

↓

FST

↓

Output: fractures

The transducer does not require a separate machine for generation. The direction of processing is simply reversed.

7.7 Handling Morphological Ambiguity

Some surface words can have more than one possible analysis.

Example 1: Treated

- treat + V + PAST
- treat + V + PARTICIPLE

The surrounding sentence helps identify the correct grammatical function.

Example 2: Monitoring

- monitor + V + ING
- monitoring + N

The word can function as a verb form or a noun.

The transducer can return multiple possible analyses. A downstream component can use sentence structure and context to select the most appropriate interpretation.

7.8 Importance of Morphological Parsing

Morphological parsing helps the hospital search system:

1. Identify the root form of clinical words.
2. Connect singular and plural forms.
3. Recognise different verb forms.
4. Improve retrieval accuracy.
5. Handle multiple analyses of ambiguous words.

Conclusion:

A finite-state transducer provides a systematic method for analysing and generating morphological word forms. It improves search by connecting different surface forms with their underlying lexical representations.

TASK 8 – ORTHOGRAPHIC RULES AND TWO-LEVEL MORPHOLOGY (4 MARKS)

8.1 Introduction

Orthographic rules describe spelling changes that occur when affixes are added to word stems.

A simple lexicon transducer cannot always produce the correct spelling by directly joining a stem and a suffix.

For example:

- Admit + ing = Admitting
- Study + ed = Studied
- Examine + ing = Examining

These forms require spelling transformations.

8.2 Why a Lexicon Transducer Alone Is Insufficient

Consider the following examples:

Stem	Suffix	Incorrect Direct Concatenation	Correct Form
Admit	-ing	Admitting with an incorrect unprocessed boundary	Admitting
Study	-ed	Studyed	Studied
Examine	-ing	Examineing	Examining
Stop	-ed	Stoped	Stopped

A lexicon transducer stores the relationship between stems and grammatical features. Orthographic rules are required to generate the correct surface spelling.

8.3 Orthographic Rules

The following rewrite rules describe common English spelling changes.

Rule 1 – Final E Deletion

$e \rightarrow \emptyset$ / _ ing

The final silent e is deleted before the suffix -ing.

Example:

Examine + ing → Examining

Rule 2 – Y-to-I Conversion

y → i / consonant _ ed

A final y changes to i when preceded by a consonant and followed by -ed.

Example:

Study + ed → Studied

Rule 3 – Consonant Doubling

C → CC / stressed short vowel _ suffix

The final consonant is doubled in suitable consonant-vowel-consonant words.

Example:

Admit + ing → Admitting

Rule 4 – Sibilant -es Addition

+ s → + es / sibilant _

Words ending in sibilant sounds take -es in relevant plural and third-person forms.

Example:

Watch + s → Watches

Rule 5 – Regular Past Tense Formation

+ ed → + d / e _

When a stem ends in e, only d is added.

Example:

Examine + ed → Examined

These rules are simplified descriptions. A practical implementation must also account for stress, exceptions and irregular forms.

8.4 Transducer Composition

The morphological system contains multiple transducers that work together.

Cascade Diagram:

Lexical Representation
↓
Lexicon Transducer
↓
Morphological Feature Processing

↓
Orthographic Rule Transducers
↓
Surface Word Generation

For example:

admit + V + ING

↓

Lexicon Transducer

↓

admit + ing

↓

Consonant Doubling Rule

↓

admitting

The order of rules is important because one spelling change may affect the input required by another rule.

8.5 Sequential Cascade and Two-Level Morphology

Sequential Cascade	**Two-Level Morphology**
Applies rules one after another.	Applies spelling correspondences in parallel.
Intermediate forms are produced.	Lexical and surface forms are related directly.
Rule order can affect the result.	Uses contextual constraints to control correspondences.
Easier to understand as separate stages.	Provides a unified morphological representation.

The sequential cascade is easier to implement and debug when the number of rules is limited.

Two-level morphology is useful for larger morphological systems because it represents relationships between lexical and surface characters through context-sensitive constraints.

8.6 Testing Orthographic Rules

The following examples can be used as a test set.

No.	Lexical Input	Expected Output
1	Treat + ed	Treated
2	Treat + ing	Treating
3	Admit + ed	Admitted
4	Admit + ing	Admitting
5	Study + ed	Studied
6	Study + es	Studies
7	Examine + ing	Examining
8	Examine + ed	Examined
9	Stop + ed	Stopped
10	Stop + ing	Stopping
11	Monitor + ed	Monitored
12	Monitor + ing	Monitoring
13	Watch + es	Watches
14	Dress + es	Dresses
15	Box + es	Boxes
16	Carry + ed	Carried
17	Play + ed	Played
18	Open + ed	Opened
19	Fix + es	Fixes
20	Call + ed	Called

These are expected outputs for rule testing. The actual implementation must be executed to record which cases pass or fail.

Conclusion:

Orthographic rules are necessary to produce correctly spelled morphological forms. Combining lexicon transducers with spelling-rule transducers helps the system analyse and generate clinical words accurately.

TASK 9 – THE PORTER STEMMER (5 MARKS)

9.1 Introduction

The Porter Stemmer is a rule-based stemming algorithm used to reduce English words to shorter root-like forms.

It removes common suffixes without requiring a dictionary.

For example:

- Infections → infect
- Connected → connect
- Monitoring → monitor

Stemming is useful for information retrieval because different forms of a word can be grouped under a common representation.

9.2 Consonants and Vowels

The Porter algorithm classifies letters into vowels and consonants.

Vowels are:

a, e, i, o, u

The letter y is treated as a consonant when it occurs at the beginning of a word and otherwise depends on the preceding character.

For example:

- Treat: t is consonant, r is consonant, e is vowel, a is vowel, t is consonant.
- Patient: p is consonant, a is vowel, t is consonant, i is vowel, e is vowel, n is consonant, t is consonant.

9.3 Measure m of a Word

The measure m represents the number of vowel-consonant sequences in the stem.

A word can be represented using the pattern:

[C](VC)^m[V]

Where:

- C represents a consonant sequence.
- V represents a vowel sequence.

- m represents the number of VC combinations.

For example:

The word "trouble" can be represented approximately as:

CVCVCV

The VC sequences are counted to determine its measure.

The measure is used to decide whether certain suffix-removal rules can be applied.

9.4 Porter Conditions

The Porter algorithm uses several conditions.

1. Condition *v*

Checks whether a stem contains a vowel.

**2. Condition d*

Checks whether a stem ends in a double consonant.

Example:

-tt

-ss

-ll

**3. Condition o*

Checks whether a stem ends in a consonant-vowel-consonant pattern where the final consonant is not w, x or y.

These conditions help control suffix removal.

9.5 Five Steps of the Porter Stemmer

Step 1 – Removing Plural and Verb Endings

Examples:

- caresses → caress
- ponies → poni
- agreed → agree
- relational → relation

Step 2 – Replacing Longer Suffixes

This step replaces certain suffixes with shorter forms.

Examples:

- relational → relate
- conditional → condition
- digitizer → digitize

Step 3 – Removing Additional Suffixes

This step removes suffixes such as:

- -icate
- -ative
- -alize
- -iciti

Examples:

- triplicate → triplic
- formalize → formal

Step 4 – Removing More Suffixes

Suffixes such as:

- -al
- -ance
- -ence
- -er
- -ic
- -able
- -ment
- -ous

are removed when the measure condition is satisfied.

Step 5 – Final Adjustments

This step removes a final e under the appropriate conditions and reduces certain double consonants.

Examples:

- probate → probat
- rate → rate
- controll → control

9.6 Worked Clinical Examples

The following table shows representative Porter-style stemming examples.

Original Word	Stemmed Form
Infections	infect
Connected	connect
Monitoring	monitor
Treatment	treatment
Fractures	fractur
Relational	relat

The Porter Stemmer does not always produce a valid dictionary word. It produces a stem suitable for matching.

9.7 Difference Between Stemming and Lemmatisation

Stemming	Lemmatisation
Removes suffixes using rules.	Identifies the dictionary form of a word.
Does not require a full lexicon.	Requires vocabulary and morphological information.
May produce incomplete words.	Produces valid lexical forms when supported.
Faster and simpler.	More linguistically accurate.
Example: fractures → fractur.	Example: fractures → fracture.

A morphological transducer is more suitable for the hospital system when accurate medical vocabulary is important.

9.8 Over-Stemming and Under-Stemming

Over-Stemming:

Over-stemming occurs when unrelated words are reduced to the same stem.

Example:

- University
- Universal

If an aggressive stemmer groups unrelated words, search results may become misleading.

In clinical records, confusing hypertension and hypotension would be especially dangerous because they represent opposite conditions.

Under-Stemming:

Under-stemming occurs when related words are not reduced to a common form.

Example:

- Fracture
- Fractures

If these forms are not connected, a search may miss relevant records.

The error rates must be calculated using the 200 manually annotated word forms required by the assignment.

9.9 Retrieval Evaluation

The assignment requires comparing three search conditions.

Evaluation Condition	Description
No Stemming	Searches using the exact input word.
Porter Stemming	Searches using the Porter stem.
Full Morphological Analysis	Searches using stems and grammatical features from the FST.

The following metrics are used:

Precision = TP / (TP + FP)

Recall = TP / (TP + FN)

F1 Score = 2 × Precision × Recall / (Precision + Recall)

The actual precision, recall and F1 scores must be obtained by running the 40 clinician queries against the prepared corpus.

Conclusion:

The Porter Stemmer provides a simple method of reducing different word forms. However, it may produce incomplete stems and can sometimes combine clinically different words. Full morphological analysis is preferable when accuracy and preservation of medical meaning are important.

TASK 10 – IMPLEMENT AND TEST THE PIPELINE (3 MARKS)

10.1 Introduction

The proposed system is implemented using Python. Python provides regular expressions, string processing and data structures that are suitable for developing the initial NLP pipeline.

The implementation uses the standard library and NLTK for stemming.

The complete pipeline contains:

1. Text preprocessing.
2. Tokenisation.
3. Clinical entity recognition.
4. Text normalisation.
5. Morphological analysis.
6. Porter stemming.
7. Search preparation.

10.2 Software Requirements

Requirement	Purpose
Python 3	Main programming language
VS Code	Development environment
NLTK	Porter Stemmer

re	Regular expression processing
csv	Reading clinical records
time	Measuring execution time
collections	Storing and counting word forms

Install NLTK using:

pip install nltk

10.3 Python Implementation

```
import re
import csv
import time
from collections import defaultdict
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()

# 1. Tokenisation
token_pattern = re.compile(
    r'\d+(?:\.\d+)?(?:mg/dL|g/dL|mmHg|mmol/L|mg|mcg|ml|kg)?'
    r'|[A-Za-z]+(?:-[A-Za-z]+)*'
    r'|[^\s]'
)

def tokenize(text):
    return token_pattern.findall(text)

# 2. Clinical Entity Patterns
```

```
patterns = {
    "Patient ID": r'\bPAT[0-9]{4,8}\b',
    "Date": r'\b(?:0[1-9]|[12][0-9]|3[01])[-/](?:0[1-9]|1[0-2])[-/]20[0-9]{2}\b',
    "Drug Strength": r'\b[A-Za-z][A-Za-z-]*\s+[0-9]+(?:\.[0-9]+)?\s*(?:mg|g|mcg|ml)\b',
    "Frequency": r'\b(?:OD|BD|TDS|QID|BID)\b',
    "Blood Pressure": r'\b[0-9]{2,3}/[0-9]{2,3}\s*mmHg\b',
    "Temperature": r'\b(?:3[0-9](?:\.[0-9]+)?|4[0-2](?:\.[0-9]+)?)\s*°?C\b',
    "ICD Code": r'\b[A-TV-Z][0-9][A-Z0-9](?:\.[A-Z0-9]{1,4})?\b',
    "Contact": r'\b(?:\+91[- ]?)?[6-9][0-9]{9}\b',
    "Age": r'\b(?:[1-9][0-9]?|1[01][0-9]|120)\s*(?:years?|yrs?)\b',
    "Duration": r'\b[0-9]+\s*(?:hours?|days?|weeks?|months?)\b'
}

# 3. Abbreviation Dictionary
abbreviations = {
    "bp": "blood pressure",
    "hr": "heart rate",
    "hb": "haemoglobin",
    "tds": "three times daily",
    "bd": "twice daily",
    "od": "once daily",
    "icu": "intensive care unit"
}

# 4. Normalisation
def normalise(text):
    text = text.lower()
```

```
    for short, full in abbreviations.items():
        text = re.sub(
            r'\b' + re.escape(short) + r'\b',
            full,
            text
        )

    text = re.sub(
        r'\b(\d{1,2})/(\d{1,2})/(20\d{2})\b',
        lambda m: f"{m.group(3)}-{m.group(2)}-{m.group(1)}",
        text
    )

    return re.sub(r'\s+', ' ', text).strip()

# 5. Entity Extraction
def extract_entities(text):
    result = {}

    for name, pattern in patterns.items():
        result[name] = re.findall(
            pattern, text, re.I
        )

    return result
```

```
# 6. Morphological Analysis
irregular = {
    "was": ("be", "V", "PAST"),
    "were": ("be", "V", "PAST"),
    "has": ("have", "V", "3SG"),
    "had": ("have", "V", "PAST"),
    "went": ("go", "V", "PAST")
}

def analyse(word):
    word = word.lower()

    if word in irregular:
        return irregular[word]

    if word.endswith("ies") and len(word) > 3:
        return (word[:-3] + "y", "N", "PL")

    if word.endswith("ing") and len(word) > 4:
        return (word[:-3], "V", "ING")

    if word.endswith("ed") and len(word) > 3:
        return (word[:-2], "V", "PAST")

    if word.endswith("s") and len(word) > 2:
        return (word[:-1], "N", "PL")
```

```
    return (word, "UNKNOWN", "BASE")

# 7. Complete Pipeline
def process_record(text):
    start = time.time()

    tokens = tokenize(text)
    entities = extract_entities(text)
    normalised = normalise(text)

    analyses = [
        analyse(token)
        for token in tokens
        if token.isalpha()
    ]

    stems = [
        stemmer.stem(token.lower())
        for token in tokens
        if token.isalpha()
    ]

    elapsed = time.time() - start

    return {
        "tokens": tokens,
        "entities": entities,
```

```
        "normalised": normalised,
        "analyses": analyses,
        "stems": stems,
        "runtime": elapsed
    }

# 8. Test Record
record = """
PAT12345 admitted on 12/04/2026.
Patient has fever for 3 days.
BP 120/80 mmHg.
Paracetamol 500mg given TDS.
"""

result = process_record(record)

print("TOKENS:", result["tokens"])
print("ENTITIES:", result["entities"])
print("NORMALISED:", result["normalised"])
print("MORPHOLOGICAL ANALYSIS:", result["analyses"])
print("PORTER STEMS:", result["stems"])
print("RUNTIME:", result["runtime"])
```

10.4 Test Cases

Test Case	Input	Expected Behaviour
1	PAT12345	Accept patient ID
2	PAT12	Reject patient ID
3	12/04/2026	Recognise date
4	19/19/2026	Reject invalid date
5	Paracetamol 500mg	Recognise drug strength
6	120/80 mmHg	Recognise blood pressure
7	9876543210	Recognise contact number
8	Fractures	Identify fracture-related form
9	Treated	Identify past-tense form
10	Monitoring	Identify -ing form
11	Was	Retrieve irregular analysis
12	Patients	Identify plural form

10.5 Pipeline Evaluation

The complete project must be tested using the following required dataset:

Component	Required Test Data	Evaluation
Clinical Corpus	500 records	Processing coverage
Word Tokens	25,000 or more	Tokenisation
Record Types	At least 4	Coverage by document type
Regular Expressions	12 entity types	Precision and recall
Automata	4 or more	Acceptance and rejection
Lexicon	At least 300 stems	Morphological coverage
Inflectional Affixes	At least 20	Rule coverage

Derivational Affixes	At least 15	Rule coverage
Clinician Queries	40	Retrieval evaluation
Annotated Word Forms	200	Morphological accuracy

10.6 Performance Measurements

The following measurements should be recorded after executing the complete implementation.

Pipeline Stage	**Number Processed**	**Number Accepted**	**Number Rejected**	**Runtime**
Tokenisation	To be measured	To be measured	To be measured	To be measured
Entity Recognition	To be measured	To be measured	To be measured	To be measured
Normalisation	To be measured	To be measured	To be measured	To be measured
Morphological Analysis	To be measured	To be measured	To be measured	To be measured
Porter Stemming	To be measured	To be measured	To be measured	To be measured

The actual values must be collected by running the system against the prepared corpus.

10.7 Graphical Evaluation

The following graphs can be generated from the experimental results:

1. Analysis coverage against lexicon size.
2. Runtime against record length.
3. Precision and recall comparison between search methods.
4. Number of successful and failed entity recognitions.

The graphs help identify the performance and limitations of the proposed system.

Conclusion:

The Python implementation provides the basic structure of the hospital text-processing pipeline. Testing the complete system with the required corpus and annotated data is necessary to verify its accuracy, efficiency and reliability.

TASK 11 – COMPARE, RECOMMEND AND DEFEND THE NORMALISATION DESIGN (2 MARKS)

11.1 Comparison of Normalisation Techniques

Three different normalisation approaches are considered for the hospital record system.

1. No normalisation.
2. Porter Stemming.
3. Full finite-state morphological analysis.

Feature	**No Normalisation**	**Porter Stemmer**	**Full Morphological Analysis**
Processing	Exact word matching	Suffix removal	Lexical and grammatical analysis
Accuracy	Limited by exact matching	May produce incorrect stems	More linguistically controlled
Coverage	Lower for word variations	Higher for inflected forms	Higher for known lexical forms
Runtime	Low	Low	Depends on lexicon and rules
Maintenance	Low	Low	Requires regular updates
Medical Vocabulary	Limited	May damage medical terms	Can preserve medical distinctions

11.2 Recommended Design

I recommend a hybrid normalisation system combining regular expressions, finite-state morphological analysis and selective stemming.

The reasons are:

1. Regular expressions identify structured information such as patient IDs, dates and dosages.
2. Finite-state automata validate predictable formats.
3. Morphological analysis connects different forms of medical words.
4. The Porter Stemmer can be used as a fallback for selected out-of-vocabulary words.

5. The original clinical records are preserved to avoid losing important information.

11.3 Expected Accuracy and Failure Modes

The proposed system is expected to improve search consistency, but its actual accuracy depends on the quality of the corpus, lexicon and evaluation data.

Possible failure modes include:

- Unrecognised medical abbreviations.
- Misspelt drug names.
- Incorrect morphological analysis.
- Ambiguous clinical terminology.
- Unusual date and dosage formats.

For example, if the system confuses hypertension with hypotension, the clinician may receive irrelevant or misleading records.

11.4 Manual Effort and Maintenance

Building a finite-state system requires manual effort in preparing the medical lexicon, writing regular expressions, defining morphological rules and annotating test data.

As the hospital vocabulary grows, maintaining all these rules may become difficult.

A statistical or machine-learning approach becomes more economical when the dataset is sufficiently large and representative, and when manual rule maintenance becomes more expensive than training and evaluating a suitable model.

Conclusion:

A hybrid finite-state normalisation design is a suitable starting point for the hospital record system because it combines predictable rule-based processing with controlled morphological analysis. It also provides a foundation for future improvements using statistical methods.

TASK 12 – DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT THE DESIGN (8 MARKS)

12.1 Introduction

The original hospital text-processing system was designed mainly for clean, typed English clinical records. However, the new requirement introduces two additional types of input:

1. OCR-generated text from scanned handwritten medical forms.
2. Romanised regional language mixed with English.

These changes introduce spelling errors, missing characters, incorrect word boundaries and words that are not present in the original English lexicon.

Therefore, the existing system must be evaluated and modified to handle noisy and mixed-language clinical information.

A. IMPACT ANALYSIS (2 MARKS)

A.1 Changes in Input Characteristics

The new input conditions introduce several challenges.

New Input Problem	Example	Effect on Original System
Character-level OCR errors	O instead of 0	Patient IDs may not be recognised.
Character confusion	rn instead of m	Medical terms may be incorrectly identified.
Broken words	dis- charge	Tokenisation may produce incorrect words.
Lost diacritics	Regional-language words	Original spelling information may disappear.
Out-of-lexicon words	Unrecognised medical terms	Morphological analysis may fail.
Code-mixing	English and Tamil written in Roman script	English-only morphology cannot analyse every token.
Spelling variations	Paracetmol instead of Paracetamol	Exact pattern matching may fail.

A.2 Assumptions That No Longer Hold

The original system was based on four important assumptions:

1. The input consists of clean and correctly typed text.
2. The vocabulary is mostly known and available in the lexicon.
3. The language is primarily English.
4. Regular expressions and automata can identify entities through exact pattern matching.

The new requirements violate these assumptions.

The system must now process uncertain characters, unknown words and multiple languages. Therefore, exact matching alone is not sufficient.

A.3 Effect of Noise on Pattern Precision and Recall

OCR errors can reduce both precision and recall.

For example, a patient identifier written as PAT12345 may be recognised correctly. However, if OCR produces PAT12S45, the regular expression may reject it because the letter S appears in a position where a digit is expected.

Similarly, an incorrect character sequence may accidentally match a different entity pattern, producing a false positive.

The effect can be measured using:

Precision = TP / (TP + FP)

Recall = TP / (TP + FN)

The actual degradation must be measured using a noisy test dataset and compared with the clean-corpus baseline.

Interpretation:

A strict finite-state system provides reliable validation for clean inputs, but it becomes less flexible when characters are misspelt or incorrectly recognised.

B. EVALUATION OF THE ORIGINAL DESIGN (2 MARKS)

B.1 Testing New Clinical Records

The original pipeline can be evaluated using three representative records.

Record 1 – OCR-Generated Text

Input:

PAT12S45 admitted on 12/04/2026

with fev3r for 3 days.

Expected information:

- Patient ID: PAT12345
- Admission date: 12/04/2026
- Symptom: Fever
- Duration: 3 days

Failure:

The patient identifier contains S instead of 3, and fever contains an OCR character error. Exact regular expressions may fail to identify these expressions.

Record 2 – Romanised Regional Language Mixed with English

Input:

patient ku fever irundhuchu,

paracetamol 500mg kuduthen.

Expected information:

- Patient experienced fever.
- Paracetamol 500mg was administered.

Failure:

The English morphological analyser can recognise fever and paracetamol, but it cannot analyse the Romanised regional-language words using an English-only lexicon.

Record 3 – OCR and Code-Mixing Combined

Input:

PAT1234S patient ku fever

irundhuchu. Paracetmol 500mg

kuduthen.

Expected information:

- Patient identifier.
- Fever-related observation.

- Paracetamol 500mg dosage.

Failure:

The patient ID contains an incorrect character, the drug name is misspelt, and the sentence contains mixed-language expressions.

The original system may recognise some numerical information but fail to identify the complete meaning of the record.

B.2 Measuring System Degradation

The following metrics should be compared between the original clean dataset and the new noisy dataset.

Evaluation Metric	Clean Dataset	Noisy Dataset
Pattern Precision	To be measured	To be measured
Pattern Recall	To be measured	To be measured
Morphological Coverage	To be measured	To be measured
Retrieval F1 Score	To be measured	To be measured

The measurements must be obtained by running the same test cases through both versions of the system.

B.3 Alternative Techniques

Three additional techniques can improve the original system.

1. Weighted Finite-State Automata

Weighted automata assign costs or weights to different transitions. They can represent alternative matching paths and prefer more likely interpretations.

For example, an OCR character confusion between O and 0 can be assigned a lower correction cost than an unrelated character substitution.

2. Edit-Distance Matching

Edit distance measures how many character insertions, deletions and substitutions are required to transform one word into another.

For example:

Paracetmol

Paracetamol

The system can compare the misspelt word with known medical terms and suggest the closest candidate.

3. Character N-gram Language Identification

Character n-grams are sequences of consecutive characters used to identify language patterns.

They can help distinguish English words from Romanised regional-language tokens.

For example, a sentence containing English medical terms and Romanised Tamil expressions can be divided into language-specific tokens.

Conclusion:

The original design provides a useful foundation, but exact matching is insufficient for noisy and mixed-language records. Additional techniques are needed to improve recognition and coverage.

C. ADAPTATION OR REDESIGN (2 MARKS)

C.1 Recommended Approach

I recommend modifying the original system rather than completely replacing it.

The existing regular expressions, finite-state automata and morphological analyser can still process correctly written English clinical information. Additional preprocessing and language-handling components can be introduced to manage OCR errors and code-mixed text.

C.2 Revised System Architecture

The modified pipeline is as follows:

Raw Hospital Records

|

v

Input Collection

|

v

OCR Text Cleaning

|

v

Character Error Correction

|

v

Tokenisation

|

v

Language Identification

|

v

Regular Expression Entity Recognition

|

v

Finite-State Validation

|

v

Morphological Analysis

|

v

Out-of-Vocabulary Handling

|

v

Text Normalisation

|

v

Search Index Creation

|

v

Clinical Search Results

C.3 Changes in the Original Components

Component	Original Design	Modified Design
Regular Expressions	Exact pattern matching	Pattern matching with OCR correction support
Automata	Strict character transitions	Weighted matching for uncertain input
Lexicon	English clinical vocabulary	English and regional-language vocabulary
Morphological Rules	English inflection	Extended language-specific rules
Tokenisation	Clean text processing	Handles broken words and mixed-language tokens
Normalisation	Case folding and abbreviation expansion	Includes spelling correction and language-aware processing

C.4 OCR Error Correction

Edit-distance matching can identify words that are similar to known medical vocabulary.

Example:

Input: Paracetmol

Expected: Paracetamol

The system compares the input with known drug names and suggests the closest matching word.

However, correction should not be applied blindly. The original text must be preserved, and uncertain corrections should be flagged for review.

For patient identifiers, the system should not silently replace letters with digits because incorrect identifiers can lead to the wrong patient record being retrieved.

C.5 Handling Romanised Regional Languages

A separate regional-language lexicon can be maintained for frequently used clinical expressions.

Example:

Romanised Expression	English Meaning
fever irundhuchu	Had fever

vali	Pain
kuduthen	I gave
sapidunga	Please eat
thanni	Water
marundhu	Medicine
patient ku	To the patient

A language-identification component can identify whether each token belongs to English or the regional-language vocabulary.

The appropriate morphological rules can then be applied to each token.

This approach allows the system to process sentences containing both English medical terminology and Romanised regional-language expressions.

C.6 Expected Effect of the Modified Design

Parameter	**Expected Change**
Coverage	Improved because more spelling and language variations can be handled.
Precision	May decrease if incorrect OCR corrections are accepted.
Recall	Expected to improve when previously missed words are recovered.
Runtime	May increase because of edit-distance and language-identification processing.
Annotation Effort	Increases because additional vocabulary and correction examples are needed.

These are expected effects based on the design. Actual values require experimental evaluation.

Conclusion:

Modifying the existing system is more practical than replacing it completely. It preserves the advantages of finite-state processing while adding support for OCR correction, unknown words and mixed-language clinical text.

D. EVIDENCE-BASED DEFENCE (2 MARKS)

D.1 Justification of the Final Decision

The final decision is to modify the original finite-state normalisation system by introducing weighted matching, edit-distance correction and language identification.

This decision is based on the limitations observed in the original design.

1. Exact regular expressions are effective for correctly formatted patient IDs, dates and clinical measurements, but they fail when OCR changes individual characters.
2. The original English morphological analyser cannot process unknown Romanised regional-language words.
3. A fixed lexicon cannot cover every misspelt or newly introduced clinical term.
4. Edit-distance matching provides a way to identify likely spelling variations.
5. Language identification helps select the appropriate vocabulary and morphological processing method.

Therefore, retaining the existing finite-state layer while adding new components provides a balanced solution.

D.2 Remaining Limitations

Even after modification, some limitations remain.

- OCR correction may produce incorrect suggestions when handwriting is unclear.
- Romanised regional languages may have multiple spelling variations.
- The same abbreviation can have different meanings in different medical contexts.
- A morphological analyser cannot fully understand the clinical meaning of a sentence.
- Negation and relationships between different medical observations require deeper semantic analysis.
- Completely unknown words may still require manual review.

Finite-state systems are effective for finite, structured and rule-based language patterns. However, they cannot independently understand every contextual meaning or handle unrestricted language variation.

D.3 Final Recommendation

The modified hybrid system is the most suitable design for the hospital record scenario.

It combines regular expressions for structured information, finite-state automata for validation, morphological transducers for word analysis, weighted matching for uncertain characters and language identification for code-mixed input.

The original records should always be retained, and uncertain corrections should be presented for verification rather than silently accepted.

Final Conclusion:

The proposed system provides a structured approach to improving hospital record searching. Initially, regular expressions and finite-state methods are sufficient for clean clinical text. However, when OCR errors and mixed-language input are introduced, the system must become more flexible. Modifying the original design with additional correction and language-processing techniques improves its ability to handle real-world hospital records while preserving the reliability of rule-based processing.

OVERALL ASSIGNMENT CONCLUSION

This assignment explores the use of Natural Language Processing techniques for building a text normalization and search layer for a hospital record system.

Regular expressions are used to identify structured information such as patient identifiers, dates, drug dosages and laboratory values. Finite-state automata validate these patterns, while morphological analysis connects different forms of clinical words.

The Porter Stemmer provides a simple method for reducing word variations, whereas finite-state morphological parsing provides more controlled linguistic analysis. Orthographic rules help generate correctly spelled word forms.

The implementation section presents a Python-based pipeline for tokenisation, entity recognition, normalisation and stemming. The final design challenge demonstrates why additional techniques are necessary when processing OCR-generated text and Romanised regional-language expressions.

Overall, the assignment shows that combining rule-based NLP techniques with suitable error-correction and language-identification methods can improve clinical text processing and information retrieval. It also highlights the importance of preserving original medical information and validating every transformation before using the processed data for clinical searching.